Radio

Live channel messages meet relational queries.

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL radio FROM query_farm;  LOAD radio FROM query_farm;
```

01 At a glance

Subscribe to WebSocket or Redis Pub/Sub endpoints, query buffered messages, and queue replies from DuckDB. The subscription URL identifies the feed throughout its lifecycle; a loopback:// feed needs no server, for tests and offline work.

Live channel → Memory buffer → DuckDB rows

02 Subscription lifecycle

`CALL radio_subscribe(url)`

Open a feed; receive_message_capacity tunes its buffer size.

`radio_subscriptions()`

Inspect feeds: transmit counters plus received_messages_queued and _dropped_unseen.

`radio_received_messages()`

Read buffered messages across all subscriptions.

`radio_subscription_received_messages(url)`

Read the buffer for one subscription URL.

`CALL radio_listen(first, wait)`

Wait for arrivals, bounded by an INTERVAL timeout.

`CALL radio_transmit_message(…)`

Queue URL, channel, BLOB payload, attempt count and retry interval.

`CALL radio_unsubscribe(url)`

Close a feed using the exact subscription URL.

03 Keep in mind

A buffer is not durable storage

Subscriptions and messages live in process memory. Capacity limits evict older messages, counted in received_messages_dropped_unseen; process exit discards state. Persist records you need in a DuckDB table, deduplicating on subscription_id and message_id.

04 Subscribe, wait and inspect

REQUIRES SETUP

```
SET VARIABLE feed = 'ws://localhost:8765/';
CALL radio_subscribe(getvariable('feed'));
CALL radio_listen(true, INTERVAL '5 seconds');
SELECT receive_time, decode(message) AS text
FROM radio_received_messages()
WHERE subscription_url = getvariable('feed');
```

Requires a WebSocket endpoint on localhost:8765 that emits UTF-8 text messages.

05 Queue a reply and inspect delivery

REQUIRES SETUP

```
CALL radio_transmit_message(
  getvariable('feed'), 'main',
  '{"type":"ack"}'::BLOB,
  3, INTERVAL '500 milliseconds'
);
SELECT transmit_successes, transmit_failures
FROM radio_subscriptions();
```

Poll until delivery completes before calling radio_unsubscribe.

06 Test offline with loopback://

```
CALL radio_subscribe('loopback://t');
CALL radio_transmit_message('loopback://t',
  'orders', '{"value":43}', 3,
  INTERVAL '10 seconds');
CALL radio_listen(true, INTERVAL '2 seconds');
SELECT channel, decode(message) AS text
FROM radio_subscription_received_messages(
  'loopback://t');
```

The transmitted message returns to the same feed: orders, {"value":43}.

07 In practice

Payloads and delivery

message is a BLOB; use decode(message) for UTF-8 text. Transmit calls queue work, so check counters for delivery progress. WebSocket and Redis URLs are refused while enable_external_access is false; loopback:// still works.

Full docs & examples ↗ Source ↗